

Generative AI: retrieval and evaluation

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build `nw/policy`, Project 3: heading-aware chunks that carry their metadata, hybrid retrieval with fusion and a reranker, answers whose citations are checked in code, three refusal paths, and a harness that scores retrieval and generation apart and gates regressions in CI.

Where we are

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

Projects 1 and 2 classify. Nothing yet answers a question. Project 3 does, from twenty-eight policy documents, and the agent in the next part calls it as `search_policies`.

Today, step by step

- 1 Read the corpus and the contract
- 2 Tokenise sensitive fields before anything is indexed
- 3 Split on headings, carry the metadata
- 4 Hybrid retrieval with fusion, filters, and a reranker
- 5 Answer with citations enforced in code, or refuse
- 6 The harness, retrieval and generation apart
- 7 The regression gate
- 8 Change one variable
- 9 The service, session check, commit
- 10 Argue with the judge (optional)

The corpus and the set

28 documents: three superseded by a newer version with different numbers, four internal.

77 golden cases, 17 of them must refuse, including questions whose only answer sits in an internal document.

Structured output is the integration contract

Mechanism	Fails when	Portable
Vendor JSON mode	long outputs, schemas with nested optionals	no
Tool calling as a schema	the model decides not to call the tool	no
Schema in the prompt, validation in code, one repair	rarely, and visibly	every vendor

Every answer and every judgement today is a validated object from `client.structured`. Nothing downstream ever reads prose.

Chunking is an engineering decision

Strategy	Keeps	Loses	Use when
Fixed windows	nothing to configure	headings, section boundaries, metadata	unstructured prose
Heading-aware	section, title, metadata per chunk	nothing important on structured docs	policies, runbooks, contracts
Sentence windows with overlap	local context	document structure	long narrative text

A chunk here is one heading's text, split only when too long, carrying document, section, effective date, audience and `superseded_by`. That metadata makes "prefer the current SLA" a filter, not a hope.

Sensitive values never reach the index, the prompt or the log

```
"Contact ops@bluefreight.example  
about account NW-11498"
```

```
"Contact [EMAIL_1]  
about account [ACCOUNT_1]"
```

Same value, same token within one text. The model can still say "the customer at [ACCOUNT_1] asked twice".

Redaction runs inside `chunk_document`

Once step 2 passes, no chunk anywhere can carry an email, account id, card, phone or key, and a test proves it end to end.

The phone pattern requires a country code so that `2025-03-01` and `99.95` survive. Dates and percentages are the answers.

Dense finds paraphrases, BM25 finds exact terms

Dense

Finds "how fast do you respond" when the document says "acknowledgement time".

Blurs 99.95 , sla-2023 , P0 .

BM25

Finds the exact term every time.

Misses every paraphrase.

Reciprocal rank fusion merges the two ranked lists: each list adds $1 / (k + \text{rank})$ to a chunk's score, $k = 60$, no weight to tune.

Fuse first, filter second, rerank last

- 1 Dense candidates, 30 of them
- 2 Lexical candidates, fused with `rrf`
- 3 Filter: drop superseded when `current_only`, drop internal when the audience is `customer`
- 4 Cross-encoder reranks the top `max(3k, 10)` of the pool
- 5 Return the top `k`

Why the filter comes after retrieval

A superseded document may be the best lexical match. Filter before fusion and it still wins and gets fused in. Metadata decides, not luck.

The reranker reads query and chunk together. Most of today's quality gain, and most of its latency.

Citations are enforced in code, not requested in the prompt

- 1 Tell the model which chunk ids it may cite
- 2 Check every citation against the ids that were actually in the context
- 3 Drop invented ids and count them
- 4 No valid citation left: this is an answer from memory. Refuse it

Deterministic, so exactly 1.0

Citation validity in the harness must be 1.000. If it is not, the code is wrong, not the model.

Refusal is an outcome, not an error

Path	Reason on the response	Model called
Retrieval too weak to bother the model	<code>weak_retrieval</code>	no
The model says the context does not cover it	<code>model_unanswerable</code>	yes
No valid citation survived the check	<code>no_valid_citation</code>	yes

The duty manager rule for P0 tickets exists only in the internal escalation matrix. A customer asking gets a refusal; the same question with `audience: internal` gets an answer with a citation into `escalation-matrix`.

Evaluate retrieval and generation separately

Metric	Measures	Computed by
Recall@k	did the gold chunks make the list	deterministic
MRR	how high was the first gold chunk	deterministic
Faithfulness	is every claim supported by the cited passages	the Judge, against a rubric
Citation validity	were the citations real	deterministic, must be 1.0
Refusal correctness	refused when it should, answered when it should	deterministic
Latency, cost	per case	the shared cost meter

A bad answer from good retrieval is a prompt or model problem. A bad answer from bad retrieval is a chunking or index problem. One combined score hides which one you have.

A judge is an instrument, so calibrate it before you read it

0.8

Cohen's kappa on 30 human-labelled answers, false pass rate 0

- A different, stronger model than the workhorse, a rubric, a fixed version
- Agreement about 0.90 at the 0.75 threshold, false fail about 0.19, about 0.17 USD
- Agreement alone flatters a judge that passes everything; kappa corrects for chance
- The false pass rate bounds what the gate can detect. Recalibrate when the judge model, the rubric or the threshold changes

Pay for the judge once, gate on every run

about 6 USD

one judged run of the 77 cases; that run is your baseline

```
for key in METRICS:
    if cur[key] < base[key] - max_drop:
        failures.append(key)
# citation validity < 1.0 always fails
```

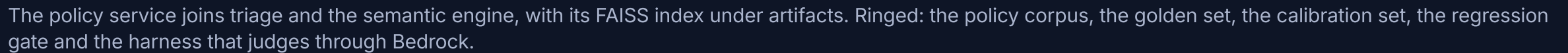
- Tolerance 0.03. Every later run is `--no-judge`, about a dollar
- Exit code 1 is what CI sees; the gate has its own on-demand workflow with OpenID Connect into your cloud account

One variable, one row on the board

Variable	Flag
Chunk size 200	<code>build_index --max-tokens 200</code> , then evaluate
k = 2, 4 or 12	<code>--k 2</code> , <code>--k 4</code> , <code>--k 12</code>
Hybrid off	<code>--no-hybrid</code>
Reranker off	<code>--no-rerank</code>
Minimum score 0.2	<code>--min-score 0.2</code>

Every run with `--no-judge`: about a dollar and a couple of minutes. Your row: which metric moved, by how much, in which direction, and what it cost in p95 latency.

GENERATIVE AI: RETRIEVAL AND EVALUATION · AWS



What exists after today, Google Cloud track

GENERATIVE AI: RETRIEVAL AND EVALUATION · GOOGLE CLOUD



Same package, same harness; Vertex AI is the endpoint for the workhorse and the judge. The provider file is the only difference.

Acceptance criteria

1. Sensitive fields never reach a chunk; dates and percentages do
2. Superseded and internal chunks are filtered by metadata, after fusion
3. Citation validity exactly 1.0 on the golden set
4. Recall@8 at or above 0.85 and correct refusal on at least 15 of the 17 must-refuse cases
5. A report with measured cost and p95 latency, and a regression gate that passes, and fails when it should

34 passed

what `make session04` prints when you are done

Questions worth asking

- You cut k from 8 to 2. Which metric moves first, and in which direction?
- The judge scores 0.5 on an answer you think is correct. Who is right, and how would you find out?
- A customer asks a question whose answer is only in the internal escalation matrix. What should the service say, and what should the log say?